

Q-Learning

PPO 是一种策略优化方法，它直接优化策略 $\pi(a|s)$ ，也就是直接学习“在某个状态下应该选择每个动作的概率”。Q-Learning 的思路不一样。它不是一开始就直接学策略，而是先学习 $Q(s,a)$ ，即在状态 s 下采取动作 a ，以后最多大概能拿到多少长期回报。

如果我知道每个状态下每个动作的 Q 值，那么我应该怎么选动作？答案很简单，在状态 s 下，选择 $Q(s,a)$ 最大的那个动作。也就是选择价值最高的动作。

在 MDP 中，智能体每一步都处在某个状态 s ，然后它选择动作 a ，环境给它奖励 r ，并转移到下一个状态 s' 。这个过程可以写成 s,a,r,s'

PPO 关注的是我怎样调整策略，让好动作概率变高，坏动作概率变低。

而 Q-Learning 关注的是我怎样估计每个动作到底值多少钱。

所以我们定义动作价值函数 $Q(s,a)$ 为当前处于状态 s ，如果我先采动作 a ，然后后面都按照最优方式行动，那么最终能得到的期望长期回报是多少。Q-Learning 学的是最优动作价值函数，也就是 $Q^*(s,a)$ ，这里的星号表示最优。

一个状态或者动作的价值，可以拆成两部分。第一部分是当前一步拿到的奖励。第二部分是下一步之后的未来价值。对于 $Q(s,a)$ 来说，如果智能体在状态 s 下采取动作 a ，它会得到即时奖励 r ，然后进入下一个状态 s' ，进入 s' 之后，智能体应该选择最好的动作。

所以用普通公式写就是：

$$Q^*(s,a) = r + \gamma \max_{a'} Q^*(s',a') \quad (1)$$

这里的 $\max_{a'} Q^*(s',a')$ 表示到了下一个状态 s' 之后，在所有可选动作 a' 中，选择 Q 值最大的那个。

如果到了下一个状态 s' ，我们想知道未来最优价值，就应该假设智能体会选最好的动作。所以不是随便选一个动作，也不是按当前策略随机选动作，而是看下一个状态 s' 中，哪个动作的 Q 值最大。于是未来价值就是 $\max_{a'} Q(s',a')$ 。

现在问题来了。

如果我们已经知道所有 $Q^*(s,a)$ ，那当然好办。在每个状态下直接选择 Q 值最大的动作就行。但现实中，我们一开始并不知道 $Q^*(s,a)$ ，所以 Q-Learning 的目标是通过不断和环境交互，一点一点把 $Q(s,a)$ 学准。

刚开始， Q 表可以全都初始化为 0。

比如： $Q(s_1,a_1) = 0; Q(s_1,a_2) = 0; Q(s_2,a_1) = 0; Q(s_2,a_2) = 0$ ，然后智能体不断尝试动作，从环境中得到奖励，再用这些经验更新 Q 值。这个过程就是 Q-Learning。

假设智能体经历了一步交互：当前状态是 s ；采取动作 a ，得到奖励 r ，进入下一个状态 s' ，这一步经验可以写成 (s,a,r,s') ，现在我们要用这条经验更新 $Q(s,a)$ 。

根据 Bellman 最优思想， $Q(s,a)$ 的理想目标应该是 $r + \gamma \max_{a'} Q(s',a')$ ，这叫做 TD target，也就是时序差分目标。即当前这个 $Q(s,a)$ 应该往这个目标靠近。也就是说，旧的 $Q(s,a)$ 可能不准。新的目标告诉我们更合理的估计应该是 $r + \gamma \max_{a'} Q(s',a')$ ，所

以我们要让 $Q(s,a)$ 向这个目标移动。

既然目标是 $r + \gamma \max Q(s',a')$ ，那为什么不直接令 $Q(s,a) = r + \gamma \max Q(s',a')$ ，而是要慢慢更新？原因是环境可能有随机性。同一个状态 s 下采取同一个动作 a ，不一定每次得到相同奖励，也不一定每次进入相同状态。如果只用一次经验就完全覆盖原来的 Q 值，估计会非常不稳定。所以 Q-Learning 使用一种“慢慢靠近”的更新方式。

设学习率为 α 。 α 在 0 到 1 之间。

如果 α 很大，说明更相信新经验。

如果 α 很小，说明更新比较保守。

更新前我们需要知道旧的 Q 值与当前新的 Q 值之间的误差，也就是

$$\text{TD error} = r + \gamma \max Q(s',a') - Q(s,a) \quad (2)$$

所以 Q-Learning 的完整更新公式是：

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max Q(s',a') - Q(s,a)] \quad (3)$$

Q-Learning 是 off-policy 方法，中文可以叫“离策略方法”。

什么叫 off-policy？意思是智能体实际采样时用的行为策略，和它学习的目标策略，可以不是同一个策略。

在 Q-Learning 中，智能体探索环境时，可能不是永远选择当前 Q 值最大的动作。它可能会为了探索，随机选择一些动作。但是它更新 Q 值时，用的是 $\max Q(s',a')$ ，也就是假设未来会选择最优动作。

所以行为时，它可能会探索；学习时，它学习的是最优贪心策略，这就是 off-policy。

如果智能体每次都选当前 Q 值最大的动作，会有一个问题：

一开始 Q 值是不准的。如果智能体过早相信错误的 Q 值，就可能永远不去尝试更好的动作。比如某个动作一开始看起来一般，但是多试几次之后可能发现它很好，如果智能体从来不尝试它，就永远不知道它好。所以 Q-Learning 需要探索。

最常见的探索方式是 ϵ -greedy，中文可以叫 ϵ -贪心策略。

策略为：大多数时候选择当前 Q 值最大的动作，少数时候随机选择动作。比如 $\epsilon = 0.1$ 就是 90% 的时候选择当前看起来最好的动作；10% 的时候随机探索。这样智能体既能利用已有经验，又能尝试新的可能。

现在我们把 Q-Learning 的训练流程讲一遍。

1. 初始化 Q 表，如果状态和动作数量有限，可以建立一个表。表中每一项是一个 $Q(s,a)$ 。刚开始可以全部设为 0，也可以设成小随机数。
2. 智能体观察当前状态 s 。
3. 根据 ϵ -greedy 策略选择动作 a 。大多数时候选择 Q 值最大的动作，少数时候随机探索。
4. 执行动作 a ，环境返回奖励 r ，并转移到下一个状态 s'
5. 计算 TD target: $\text{TD target} = r + \gamma \max Q(s',a')$
6. 计算 TD error: $\text{TD error} = \text{TD target} - Q(s,a)$
7. 更新 $Q(s,a)$: $Q(s,a) \leftarrow Q(s,a) + \alpha \times \text{TD error}$

8. 把当前状态更新为 s' 。
9. 继续下一轮。

这个过程不断重复， Q 表会逐渐变得准确。最终，在每个状态下选择 Q 值最大的动作，就可以得到一个较好的策略。

在 MDP 课中，我们讲过价值迭代。

价值迭代的更新形式大概是 $V(s)$ 根据所有可能的下一状态和转移概率进行更新，它需要知道环境模型，也就是说，它要知道采取动作 a 后，转移到每个状态 s' 的概率是多少。

但是 Q-Learning 不需要提前知道环境转移概率。它只需要真实交互得到样本： s, a, r, s' 。

所以 Q-Learning 是 model-free 方法，中文叫“无模型方法”，即智能体不需要知道环境的转移概率模型。它不用知道 $P(s'|s, a)$ 。它只要不断试，就能学。

即使我不知道环境规律，只要我能和环境互动，我就可以通过经验学习最优动作价值。

讲一讲 Q-Learning 和 SARSA 的区别

SARSA 的名字来自一条经验序列： s, a, r, s', a'

SARSA 的更新目标是 $r + \gamma Q(s', a')$ ，注意这里用的是实际选择的下一个动作 a' 。

而 Q-Learning 的更新目标是 $r + \gamma \max Q(s', a')$ ，它用的是下一个状态中最大的 Q 值

所以 SARSA 学的是按照当前行为策略实际走下去，会得到什么结果。

Q-Learning 学的是假设以后都选最优动作，会得到什么结果。

所以 SARSA 是 on-policy，也就是同策略。

Q-Learning 是 off-policy，也就是离策略。

更直白地说：

SARSA：我实际下一步怎么走，就按这个来学习。

Q-Learning：不管我实际下一步怎么走，我都按最优动作来学习。

Q-Learning 的更新目标来自 Bellman 最优方程，即：

$$Q^*(s, a) = r + \gamma \max_{a'} Q^*(s', a') \quad (4)$$

Q-Learning 每次更新，都是让当前 $Q(s, a)$ 朝着这个最优方程的右边靠近。只要智能体有足够多的探索，每个状态动作对都能被访问到足够多次，并且学习率设置合理，那么 Q 值会逐渐逼近最优 Q 值。

等 Q 值接近 Q^* 后，策略就很简单，即每个状态下选择 Q 值最大的动作。

也就是 $\pi(s) = \operatorname{argmax}_a Q(s, a)$ 。

所以 Q-Learning 先学 Q ，再由 Q 得到策略。

虽然 Q-Learning 很经典，但它也有明显局限。如果状态很多，动作也很多，那么 Q 表会非常大。比如状态有一百万个，动作有十个，那么 Q 表就有一千万个数。这在复杂任务中很难维护。

第二个问题是不能直接处理连续状态。如果状态是图像、语音、机器人关节角度这种连续高维输入，用表格记录 $Q(s, a)$ 就不现实。

第三个问题是泛化能力弱。表格 Q-Learning 中，状态 s_1 学到的经验不能很好泛化到类似状态 s_2 。所以后来人们引入神经网络来近似 Q 函数。这就发展出了 DQN。

DQN 的全称是 Deep Q-Network，中文可以叫深度 Q 网络。

它的思想是不用表格存 $Q(s, a)$ ，而是用神经网络来近似 Q 函数，输入状态 s ，输出每个动作的 Q 值。

DQN 本质上还是 Q-Learning，只是把 Q 表换成了神经网络。所以可以说 Q-Learning 是 DQN 的基础，如果不理解 Q-Learning，就很难真正理解 DQN。

现在我们把 Q-Learning 和上一节 PPO 联系起来。

PPO 是策略优化方法，它直接学习策略 $\pi(a|s)$ ，也就是直接学习每个动作的概率。

Q-Learning 是价值学习方法，它学习的是 $Q(s, a)$ ，也就是每个动作的价值。